



UNSW
SYDNEY

COMP4141

Theory of Computation

Lecture 8: Algorithms for CFLs

Outline

Decision Problems

Algorithms for Regular Languages

Algorithms for Context-Free Languages

End Matter

Common questions about languages

We have seen a number of ways of representing languages in a finite way (automata, regular expressions, grammars). This lets us address several questions algorithmically:

- Emptiness: Given a representation, is the associated language empty?
- Universality: Given a representation, is the associated language universal?
- Word/Acceptance problem: Given a word w and a representation of a language L , is $w \in L$?
- Language equivalence: Given representations of two languages, L_1 and L_2 , is $L_1 = L_2$?
- Language inclusion: Given representations of two languages, L_1 and L_2 , is $L_1 \subseteq L_2$?

Decision Problems

When we can phrase a question we want answered as a Yes/No question this is known as a **Decision Problem**.

For example:

NFA EMPTINESS

Input: An NFA N

Question: Is $L(N) = \emptyset$

Outline

Decision Problems

Algorithms for Regular Languages

Algorithms for Context-Free Languages

End Matter

Deciding emptiness of a regular language

How to decide emptiness of a regular language L depends on its representation, of which we've met a few.

NFA EMPTINESS

Input: An NFA N

Question: Is $L(N) = \emptyset$?

REGEXP EMPTINESS

Input: A regular expression R

Question: Is $L(R) = \emptyset$?

NFA Emptiness

When L is given as $L(A)$ of an NFA $(Q, \Sigma, \delta, q_0, F)$ (or DFA, ϵ -NFA) is an exercise in graph reachability: Is there a final state that can be reached from the initial state?

This can be done by a depth-first search in time linear in the number of edges and vertices. (But note there could be $|Q|^2$ edges.)

RegExp Emptiness

When L is given as $L(R)$ of a regular expression R then we can *abstract* inductively as follows:

Base: $L(\emptyset) = \emptyset$, $L(\epsilon) \neq \emptyset$, and $L(a) \neq \emptyset$.

Induction:

$L(R_1 \cup R_2) = \emptyset$ iff $L(R_1) = L(R_2) = \emptyset$.

$L(R_1 \cdot R_2) = \emptyset$ iff $L(R_1) = \emptyset$ or $L(R_2) = \emptyset$.

$L(R_1^*) \neq \emptyset$.

$L((R_1)) = \emptyset$ iff $L(R_1) = \emptyset$.

This implies that $L(R) = \emptyset$ can be decided in $O(|R|)$ time.

The word problem for regular languages

Like the emptiness problem, the word problem depends on the representation.

DFA ACCEPTANCE

Input: A DFA D and a word w

Question: Is $w \in L(D)$?

NFA ACCEPTANCE

Input: An NFA N and a word w

Question: Is $w \in L(N)$?

Word problems

DFA: easy, just feed the word to the automaton.

NFA: marking algorithm:

On input $w = a_1 \dots a_{|w|}$

- 1 Set of marks $M := \{q_0\}$.
- 2 For $i = 1$ to $|w|$ do
 $M := \bigcup_{q \in M} \delta(q, a_i)$
- 3 Return whether $F \cap M \neq \emptyset$.

Others: translate to an equivalent DFA.

The inclusion problem for regular languages

DFA LANGUAGE INCLUSION

Input: Two DFAs A_1 and A_2

Question: Is $L(A_1) \subseteq L(A_2)$?

The inclusion problem for regular languages

DFA LANGUAGE INCLUSION

Input: Two DFAs A_1 and A_2

Question: Is $L(A_1) \subseteq L(A_2)$?

Algorithm:

- 1 Construct a DFA recognising $L(A_1) \cap \overline{L(A_2)}$
- 2 Check for emptiness

The inclusion problem for regular languages

DFA LANGUAGE INCLUSION

Input: Two DFAs A_1 and A_2

Question: Is $L(A_1) \subseteq L(A_2)$?

Algorithm:

- 1 Construct a DFA recognising $L(A_1) \cap \overline{L(A_2)}$
- 2 Check for emptiness

Other representations: Translate to equivalent DFAs.

The equivalence problem for regular languages

DFA LANGUAGE EQUIVALENCE

Input: Two DFAs A_1 and A_2

Question: Is $L(A_1) = L(A_2)$?

The equivalence problem for regular languages

DFA LANGUAGE EQUIVALENCE

Input: Two DFAs A_1 and A_2

Question: Is $L(A_1) = L(A_2)$?

Algorithm:

- 1 Check if $L(A_1) \subseteq L(A_2)$
- 2 Check if $L(A_2) \subseteq L(A_1)$

The equivalence problem for regular languages

DFA LANGUAGE EQUIVALENCE

Input: Two DFAs A_1 and A_2

Question: Is $L(A_1) = L(A_2)$?

Algorithm:

- 1 Check if $L(A_1) \subseteq L(A_2)$
- 2 Check if $L(A_2) \subseteq L(A_1)$

Other representations: translate to equivalent DFAs.

Outline

Decision Problems

Algorithms for Regular Languages

Algorithms for Context-Free Languages

End Matter

Emptiness of CFLs

CFG EMPTINESS

Input: A CFG $G = (V, \Sigma, P, S)$

Question: Is $L(G) = \emptyset$?

Emptiness of CFLs

CFG EMPTINESS

Input: A CFG $G = (V, \Sigma, P, S)$

Question: Is $L(G) = \emptyset$?

The emptiness problem for CFGs can be decided as follows:

- 1 Mark the terminals and ϵ , as *generating*
- 2 Mark as generating all those non-terminals that have a production with only generating symbols in the RHS.
- 3 Repeat step 2 until nothing new is marked generating.
- 4 Check whether the start symbol is marked as generating.

Chomsky normal form

For many of the remaining questions, it is convenient to introduce a particularly simple class of CFGs that is still as powerful as CFGs in general.

Definition

A context free (type 2) grammar is in *Chomsky normal form* if every production is of one of the forms

- $S \rightarrow \epsilon$, or
- $A \rightarrow BC$ where B and C are not S , or
- $A \rightarrow a$.

Theorem

Any CFL is generated by a CFG in Chomsky normal form.

Constructing the Chomsky normal form

Let $G = (V, \Sigma, P, S)$ be a CFG.

Step 0 Remove S and ϵ from RHS

Step 0: Define an equivalent CFG $G_0 = (V \cup \{S_0\}, \Sigma, P_0, S_0)$ with a fresh start variable S_0 and the production $S_0 \rightarrow S$. We also remove all productions of the form $A \rightarrow \epsilon$ and patch this up by introducing new productions for every occurrence of A in a body with that occurrence removed.

(E.g., $A \rightarrow \epsilon \mid aAbA$ becomes $A \rightarrow ab \mid abA \mid aAb \mid aAbA$.)

Productions $B \rightarrow A$ are replaced by $B \rightarrow \epsilon$ unless that is one we had removed earlier.

Repeat until ϵ occurs at most in $S_0 \rightarrow \epsilon$.

Constructing the Chomsky normal form

Let $G = (V, \Sigma, P, S)$ be a CFG.

Step 0 Remove S and ϵ from RHS

Step 1 Remove terminals from RHS

Step 1: Define an equivalent CFG $G_1 = (V_1, \Sigma, P_1, S_0)$ with fresh non-terminals for terminals $V_1 = V \cup \{S_0\} \cup \{X_a \mid a \in \Sigma\}$. Here P_1 is derived from P_0 by replacing all occurrences of a terminal $a \in \Sigma$ in the body of a production by the corresponding non-terminal X_a and then adding productions $X_a \rightarrow a$.

Constructing the Chomsky normal form

Let $G = (V, \Sigma, P, S)$ be a CFG.

Step 0 Remove S and ϵ from RHS

Step 1 Remove terminals from RHS

Step 2 Remove singleton rules ($A \rightarrow B$)

Step 2: Define an equivalent CFG $G_2 = (V_1, \Sigma, P_2, S_0)$. To generate P_2 from P_1 , eliminate productions of the form $A \rightarrow B$ by

- 1 determining all derivations $A_1 \Rightarrow_{G_1} \dots \Rightarrow_{G_1} A_k \Rightarrow_{G_1} \alpha \notin V_1$ not containing repetitions of non-terminals,
- 2 drop all productions $A \rightarrow B$, and
- 3 introduce $A_1 \rightarrow \alpha$ for each of the derivations determined before.

Constructing the Chomsky normal form

Let $G = (V, \Sigma, P, S)$ be a CFG.

Step 0 Remove S and ϵ from RHS

Step 1 Remove terminals from RHS

Step 2 Remove singleton rules ($A \rightarrow B$)

Step 3 Reduce RHS to pairs of non-terminals

Step 3: Define an equivalent CFG $G_3 = (V_3, \Sigma, P_3, S_0)$. To generate P_3 from P_2 , replace all productions $A \rightarrow B_1 \dots B_n$ with $n > 2$ by $A \rightarrow B_1 C_1$, $C_1 \rightarrow B_2 C_2$, $\dots$, $C_{n-2} \rightarrow B_{n-1} B_n$ for fresh non-terminals C_i . (These are distinct for each such production.) □

Constructing the Chomsky normal form

Let $G = (V, \Sigma, P, S)$ be a CFG.

$$\begin{array}{lcl} S & \rightarrow & ASA \mid aB \\ A & \rightarrow & B \mid S \\ B & \rightarrow & b \mid \epsilon \end{array}$$

Constructing the Chomsky normal form

Let $G = (V, \Sigma, P, S)$ be a CFG.

Step 0 Remove S and ϵ from RHS

$$\begin{array}{lcl} S_0 & \rightarrow & S \\ S & \rightarrow & ASA \mid aB \mid a \\ A & \rightarrow & B \mid S \mid b \mid \epsilon \\ B & \rightarrow & b \end{array}$$

Constructing the Chomsky normal form

Let $G = (V, \Sigma, P, S)$ be a CFG.

Step 0 Remove S and ϵ from RHS

$$\begin{array}{lcl} S_0 & \rightarrow & S \\ S & \rightarrow & ASA \mid aB \mid a \mid SA \mid AS \mid S \\ A & \rightarrow & B \mid S \mid b \\ B & \rightarrow & b \end{array}$$

Constructing the Chomsky normal form

Let $G = (V, \Sigma, P, S)$ be a CFG.

Step 0 Remove S and ϵ from RHS

Step 1 Remove terminals from RHS

$$\begin{array}{lcl} S_0 & \rightarrow & S \\ S & \rightarrow & ASA \mid X_a B \mid a \mid SA \mid AS \mid S \\ A & \rightarrow & B \mid S \mid b \\ B & \rightarrow & b \\ X_a & \rightarrow & a \end{array}$$

Constructing the Chomsky normal form

Let $G = (V, \Sigma, P, S)$ be a CFG.

Step 0 Remove S and ϵ from RHS

Step 1 Remove terminals from RHS

Step 2 Remove singleton rules ($A \rightarrow B$)

$$\begin{array}{lcl} S_0 & \rightarrow & ASA \mid X_a B \mid a \mid SA \mid AS \\ S & \rightarrow & ASA \mid X_a B \mid a \mid SA \mid AS \\ A & \rightarrow & b \mid ASA \mid X_a B \mid a \mid SA \mid AS \\ B & \rightarrow & b \\ X_a & \rightarrow & a \end{array}$$

Constructing the Chomsky normal form

Let $G = (V, \Sigma, P, S)$ be a CFG.

Step 0 Remove S and ϵ from RHS

Step 1 Remove terminals from RHS

Step 2 Remove singleton rules ($A \rightarrow B$)

Step 3 Reduce RHS to pairs of non-terminals

$$\begin{aligned} S_0 &\rightarrow AA_1 \mid X_a B \mid a \mid SA \mid AS \\ S &\rightarrow AA_1 \mid X_a B \mid a \mid SA \mid AS \\ A &\rightarrow b \mid AA_1 \mid X_a B \mid a \mid SA \mid AS \\ B &\rightarrow b \\ X_a &\rightarrow a \\ A_1 &\rightarrow SA \end{aligned}$$

The word problem for CFLs

CFG ACCEPTANCE

Input: A CFG G and a word w

Question: Is $w \in L(G)$?

The word problem for CFLs

CFG ACCEPTANCE

Input: A CFG G and a word w

Question: Is $w \in L(G)$?

Naive algorithm:

- 1 Convert to Chomsky normal form
- 2 Explore all derivations (each derivation has length $2|w|$)

(Running time $O(2^n)$)

The word problem for CFLs

CFG ACCEPTANCE

Input: A CFG G and a word w

Question: Is $w \in L(G)$?

Naive algorithm:

- 1 Convert to Chomsky normal form
- 2 Explore all derivations (each derivation has length $2|w|$)

(Running time $O(2^n)$)

A better approach:

Theorem

Let $G = (V, \Sigma, P, S)$ be a CFG in Chomsky normal form. Then the CYK-algorithm decides $w \in L(G)$ in time $O(|w|^3)$ for $w \in \Sigma^$.*

The CYK-Algorithm

The CYK-algorithm (for Cocke-Younger-Kasami) works as follows. Given $w = a_1 \dots a_n$ compute for $i, j \in \{1, \dots, n\}$ the set $N_{ij} = \{ A \in V \mid A \Rightarrow_G^* a_i \dots a_j \}$ of non-terminals generating $a_i \dots a_j$. Then

$$w \in L(G) \quad \text{iff} \quad S \in V_{1n} .$$

The details are not in [Sipser2006].

The CYK-Algorithm

for CFG $G = (V, \Sigma, P, S)$ in Chomsky normal form.

“On input $w = a_1 \dots a_n$

1 for $i, j := 1$ to n do

$$N_{i,j} := \begin{cases} \{ A \in V \mid A \rightarrow a_i \in P \} & \text{if } i = j \\ \emptyset & \text{otherwise} \end{cases}$$

2 for $d := 1$ to $n - 1$ and $i := 1$ to $n - d$ do

1 $j := i + d$

2 for $k := i$ to $j - 1$ do

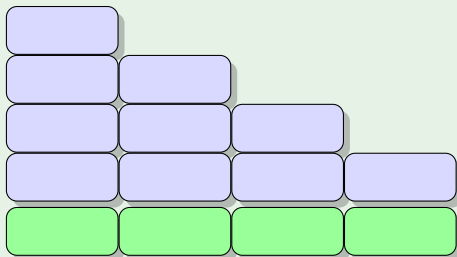
$$N_{i,j} := N_{i,j} \cup \left\{ A \in V \mid \exists B, C \left(\begin{array}{l} A \rightarrow BC \in P \wedge \\ B \in N_{i,k} \wedge C \in N_{k+1,j} \end{array} \right) \right\}$$

3 Return whether $S \in N_{1,n}$.

CYK-Algorithm Example

Example

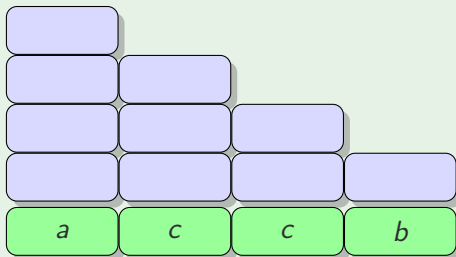
Decide if $accb \in L(G)$



CYK-Algorithm Example

Example

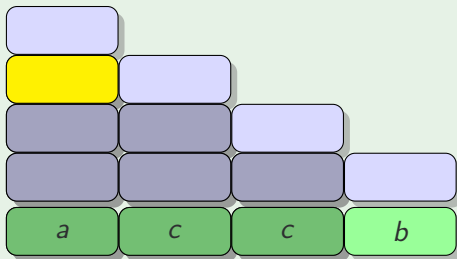
Decide if $accb \in L(G)$



CYK-Algorithm Example

Example

Decide if $accb \in L(G)$



CYK-Algorithm Example

Example

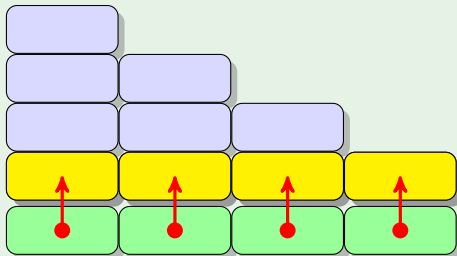
Decide if $accb \in L(G)$ where G is defined by the rules:

$S \rightarrow$

$A \rightarrow \quad a \mid b$

$B \rightarrow \quad c$

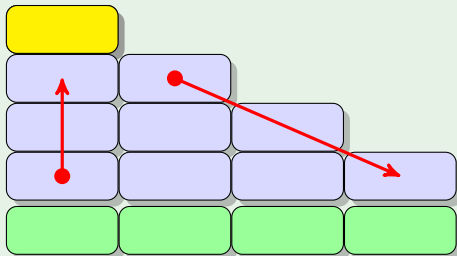
$C \rightarrow \quad \quad c$



CYK-Algorithm Example

Example

Decide if $accb \in L(G)$ where G is defined by the rules:

$$S \rightarrow AB$$
$$A \rightarrow CA$$
$$B \rightarrow BB$$
$$C \rightarrow CB \mid AB$$


CYK-Algorithm Example

Example

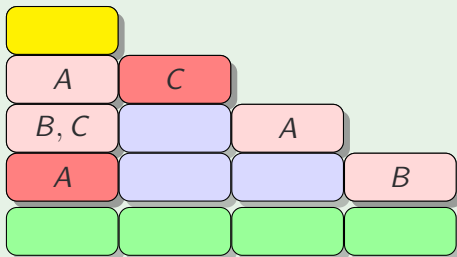
Decide if $accb \in L(G)$ where G is defined by the rules:

$$S \rightarrow AB$$

$$A \rightarrow CA$$

$$B \rightarrow BB$$

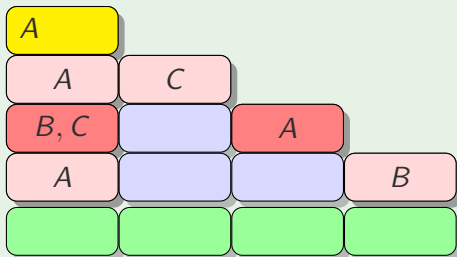
$$C \rightarrow CB \mid AB$$



CYK-Algorithm Example

Example

Decide if $accb \in L(G)$ where G is defined by the rules:

$$S \rightarrow AB$$
$$A \rightarrow CA$$
$$B \rightarrow BB$$
$$C \rightarrow CB \mid AB$$


CYK-Algorithm Example

Example

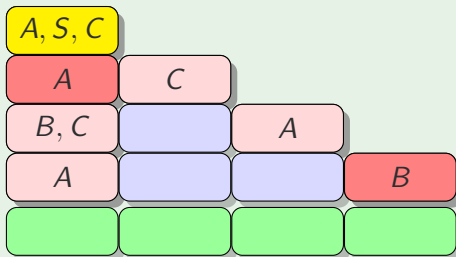
Decide if $accb \in L(G)$ where G is defined by the rules:

$S \rightarrow AB$

$A \rightarrow CA$

$B \rightarrow BB$

$C \rightarrow CB \mid AB$



CYK-Algorithm Example

Example

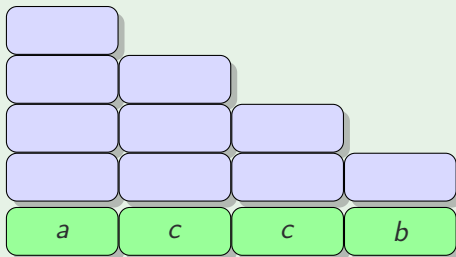
Decide if $accb \in L(G)$ where G is defined by the rules:

$$S \rightarrow AB$$

$$A \rightarrow CA \mid a \mid b$$

$$B \rightarrow BB \mid c$$

$$C \rightarrow CB \mid AB \mid c$$



CYK-Algorithm Example

Example

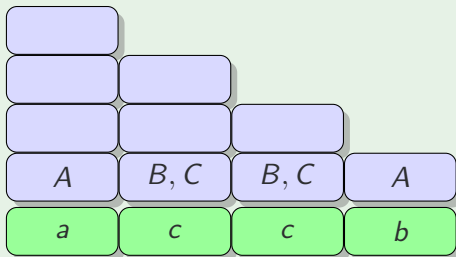
Decide if $accb \in L(G)$ where G is defined by the rules:

$$S \rightarrow AB$$

$$A \rightarrow CA \mid a \mid b$$

$$B \rightarrow BB \mid c$$

$$C \rightarrow CB \mid AB \mid c$$



CYK-Algorithm Example

Example

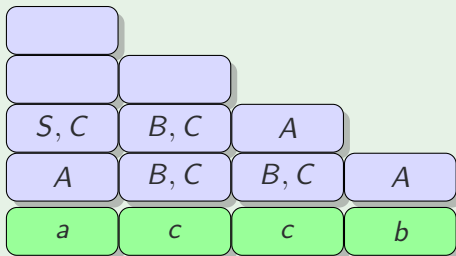
Decide if $accb \in L(G)$ where G is defined by the rules:

$$S \rightarrow AB$$

$$A \rightarrow CA \mid a \mid b$$

$$B \rightarrow BB \mid c$$

$$C \rightarrow CB \mid AB \mid c$$



CYK-Algorithm Example

Example

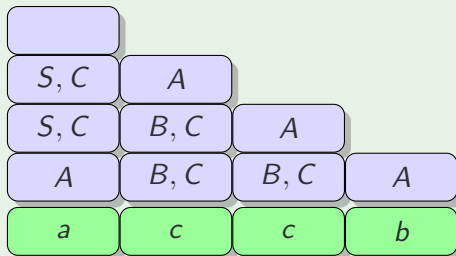
Decide if $accb \in L(G)$ where G is defined by the rules:

$$S \rightarrow AB$$

$$A \rightarrow CA \mid a \mid b$$

$$B \rightarrow BB \mid c$$

$$C \rightarrow CB \mid AB \mid c$$



CYK-Algorithm Example

Example

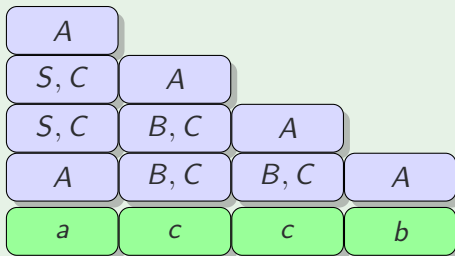
Decide if $accb \in L(G)$ where G is defined by the rules:

$$S \rightarrow AB$$

$$A \rightarrow CA \mid a \mid b$$

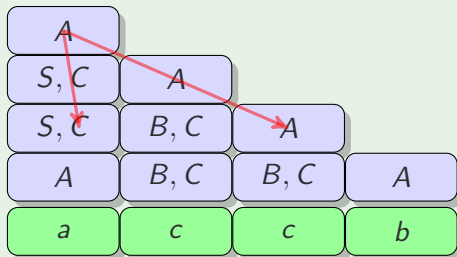
$$B \rightarrow BB \mid c$$

$$C \rightarrow CB \mid AB \mid c$$



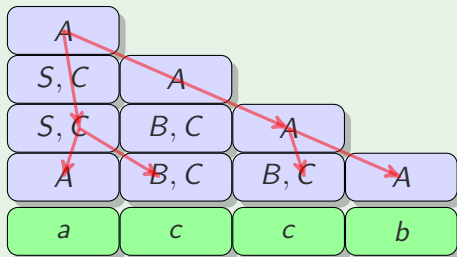
CYK-Algorithm Example

Decide if $accb \in L(G)$ where G is defined by the rules:



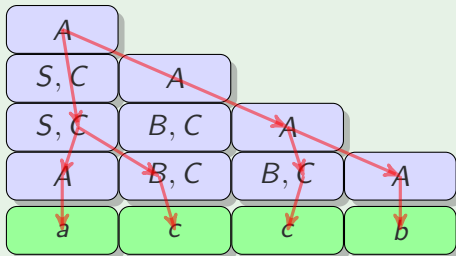
CYK-Algorithm Example

Decide if $accb \in L(G)$ where G is defined by the rules:



CYK-Algorithm Example

Decide if $accb \in L(G)$ where G is defined by the rules:



CYK-Algorithm Example

Example

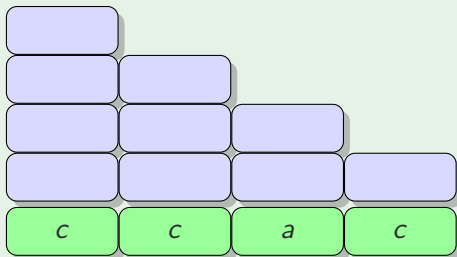
Decide if $ccac \in L(G)$ where G is defined by the rules:

$$S \rightarrow AB$$

$$A \rightarrow CA \mid a \mid b$$

$$B \rightarrow BB \mid c$$

$$C \rightarrow CB \mid AB \mid c$$



CYK-Algorithm Example

Example

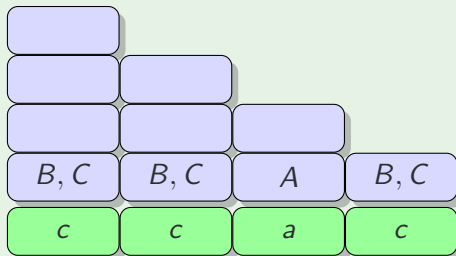
Decide if $ccac \in L(G)$ where G is defined by the rules:

$$S \rightarrow AB$$

$$A \rightarrow CA \mid a \mid b$$

$$B \rightarrow BB \mid c$$

$$C \rightarrow CB \mid AB \mid c$$



CYK-Algorithm Example

Example

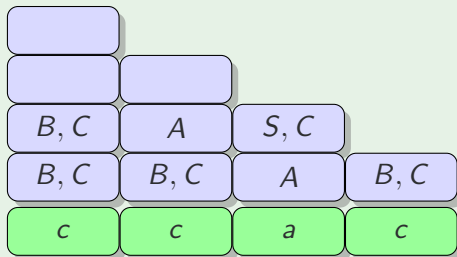
Decide if $ccac \in L(G)$ where G is defined by the rules:

$$S \rightarrow AB$$

$$A \rightarrow CA \mid a \mid b$$

$$B \rightarrow BB \mid c$$

$$C \rightarrow CB \mid AB \mid c$$



CYK-Algorithm Example

Example

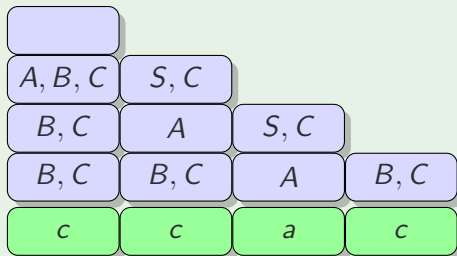
Decide if $ccac \in L(G)$ where G is defined by the rules:

$$S \rightarrow AB$$

$$A \rightarrow CA \mid a \mid b$$

$$B \rightarrow BB \mid c$$

$$C \rightarrow CB \mid AB \mid c$$



CYK-Algorithm Example

Example

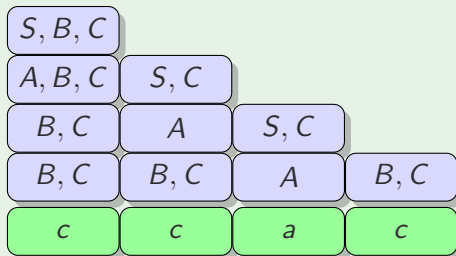
Decide if $ccac \in L(G)$ where G is defined by the rules:

$$S \rightarrow AB$$

$$A \rightarrow CA \mid a \mid b$$

$$B \rightarrow BB \mid c$$

$$C \rightarrow CB \mid AB \mid c$$



CYK-Algorithm Example

Example

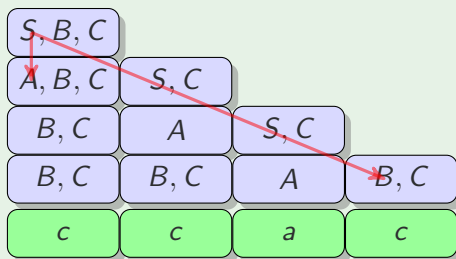
Decide if $ccac \in L(G)$ where G is defined by the rules:

$$S \rightarrow AB$$

$$A \rightarrow CA \mid a \mid b$$

$$B \rightarrow BB \mid c$$

$$C \rightarrow CB \mid AB \mid c$$



$$S \Rightarrow_G AB$$

CYK-Algorithm Example

Example

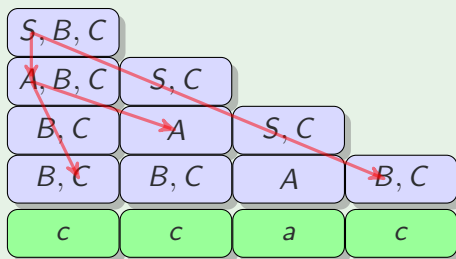
Decide if $ccac \in L(G)$ where G is defined by the rules:

$$S \rightarrow AB$$

$$A \rightarrow CA \mid a \mid b$$

$$B \rightarrow BB \mid c$$

$$C \rightarrow CB \mid AB \mid c$$



$$S \Rightarrow_G AB \Rightarrow_G CAB$$

CYK-Algorithm Example

Example

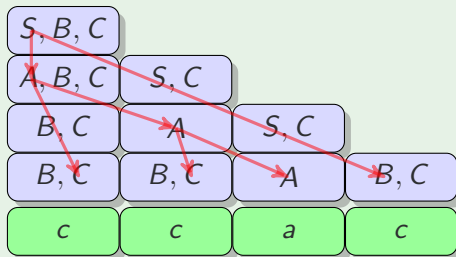
Decide if $ccac \in L(G)$ where G is defined by the rules:

$$S \rightarrow AB$$

$$A \rightarrow CA \mid a \mid b$$

$$B \rightarrow BB \mid c$$

$$C \rightarrow CB \mid AB \mid c$$



$$S \Rightarrow_G AB \Rightarrow_G CAB \Rightarrow_G CCAB$$

Preview of undecidable CFL problems

Later we'll develop a theory that allows us to prove rigorously that there are problems that *cannot be solved by any algorithm* that can be implemented as a computer program.

Such problems are called *undecidable*.

Some simple decision problems in the realm of CFLs turn out to be undecidable:

- Is a given CFG ambiguous?
- Is any CFG for a given CFL necessarily ambiguous?
- Is the intersection of two given CFLs empty?
- Are two given CFGs/PDAs equivalent?
- Does a given CFG generate all strings Σ^* ?

Outline

Decision Problems

Algorithms for Regular Languages

Algorithms for Context-Free Languages

End Matter

Weekly Feedback

I would appreciate any comments/suggestions/requests you have on this week's lectures (and tutorials).



<https://forms.office.com/r/g3LgPASave>